Carnet de Bord
Robot Auto-Équilibré STM32F4

Développement d'un robot auto-équilibré capable de se déplacer et d'évoluer vers la navigation autonome.

Principe du projet

Pourvu qu'aujourd'hui soit mieux qu'hier.

Progresser chaque jour, même légèrement, est une victoire.

Auteur : Ithiel Gabriel Djifa DENYIGBA

Projet : Robot auto-équilibré STM32F4
Version : 1.0
Début : Juillet 2026
Budget initial : 200 €

Table des matières

1 Pourquoi ce projet ? 2

2 Cahier des charges 2

3 Roadmap de développement 3

4 Journal de bord 4

5 Les composants 36

6 Notions théoriques 37

7 Architecture du robot 53

8 Les erreurs et leçons apprises 54

9 Améliorations futures 55

10 Matériel et plan d'achat 55

Conclusion 56

1. Pourquoi ce projet ?

Ce projet est un laboratoire personnel pour apprendre et mettre en pratique les notions fondamentales des systèmes embarqués, de l'automatique et de la robotique mobile.

Il doit permettre de développer des compétences solides en :

- systèmes embarqués ;
- programmation STM32 ;
- électronique appliquée ;
- contrôle moteur ;
- automatique et PID ;
- traitement du signal ;
- fusion de capteurs ;
- robotique mobile ;
- architecture logicielle ;
- gestion de projet technique.

L'objectif n'est pas seulement de construire un robot. Le robot est le résultat visible. Le véritable objectif est de progresser comme ingénieur en apprenant à concevoir, tester, corriger, documenter et améliorer un système réel.

Principe du projet

Le robot n'est pas seulement une machine. Il devient un support d'apprentissage, un terrain d'expérimentation et une preuve concrète de progression.

Pourvu qu'aujourd'hui soit mieux qu'hier.

2. Cahier des charges

Objectif général

Construire un robot auto-équilibré basé sur une carte STM32F4, capable de se stabiliser, de se déplacer et, à terme, d'évoluer vers une navigation autonome.

Version 1 – Stabilisation de base

Le robot doit :

- tenir debout en équilibre ;
- avancer ;
- reculer ;
- être contrôlé par une boucle temps réel ;
- exploiter une IMU pour estimer son inclinaison ;
- utiliser des moteurs avec encodeurs pour mesurer le mouvement.

Version 2 – Robot connecté

Ajouter :

— Wi-Fi via ESP32 ;

— télémétrie vers PC ;

— réglage des paramètres PID à distance ;

— contrôle manuel depuis une interface distante.

Version 3 – Navigation autonome

Ajouter :

— navigation d'un point A vers un point B ;

— estimation de déplacement par odométrie ;

— évitement d'obstacles ;

— planification de trajectoire simple.

Contraintes de conception

— privilégier l'apprentissage progressif ;

— valider chaque brique séparément avant intégration ;

— garder une architecture logicielle claire ;

— documenter chaque séance ;

— éviter d'ajouter des fonctionnalités avancées avant stabilisation.

3. Roadmap de développement

Phase	Objectif	Compétences travaillées
Phase 1	Prise en main STM32	GPIO, UART, Timers, PWM, interruptions, ADC
Phase 2	Briques robot	IMU, I2C, moteur, encodeur, filtre complémentaire
Phase 3	Intégration mécanique	châssis, câblage, alimentation, centre de gravité
Phase 4	Stabilisation	PID angle, boucle temps réel, tuning
Phase 5	Déplacement	vitesse, odométrie, contrôle dynamique
Phase 6	Autonomie	Wi-Fi, télémétrie, navigation, évitement d'obstacles

Principe du projet

Chaque phase est une marche. L'objectif n'est pas de sauter les étapes, mais de les valider proprement.

4. Journal de bord

Chaque séance suit ce format :

- Objectif ;
- Réalisation ;
- Problèmes rencontrés ;
- Solution trouvée ;
- Apprentissage ;
- Prochaine étape.

Séance 0 – Naissance du projet

Journal de séance

Date : Juillet 2026

Objectif :
Définir la vision du projet, concevoir son architecture et sélectionner l'ensemble du matériel nécessaire à la réalisation du robot auto-équilibré.

Réalisation :

- Définition de la vision du projet.
- Création du carnet de bord.
- Élaboration de la feuille de route de développement.
- Définition des différentes phases du projet.
- Étude des architectures possibles.
- Comparaison des différentes cartes STM32.
- Sélection des moteurs, capteurs et composants.
- Vérification de la compatibilité de tous les éléments.
- Validation de l'architecture électrique.
- Commande de l'ensemble des composants.

Matériel commandé :

- STM32 Nucleo-F446RE.
- Module IMU MPU6050.
- Driver moteur TB6612FNG.
- Deux motoréducteurs avec encodeurs.
- Batterie LiPo 2S 7,4 V – 2200 mAh.
- Chargeur LiPo avec équilibrage.
- Convertisseur Buck LM2596.
- Connecteurs Deans avec câbles silicone.
- Interrupteur ON/OFF.
- Multimètre numérique.
- Cutter de précision.
- Plaques de PVC expansé.
- Fils de câblage.
- Câbles Dupont.

Budget du projet :

200 €
Version 1

Difficultés rencontrées :

— Choisir entre plusieurs cartes STM32.
— Comprendre l'alimentation d'un robot mobile.
— Sélectionner une batterie compatible.
— Comprendre le rôle du Buck Converter.
— Choisir des moteurs adaptés au contrôle d'équilibre.
— Vérifier la compatibilité entre tous les composants.

Ce que j'ai appris :
Avant de programmer un robot, il faut d'abord le concevoir. Un bon projet commence par une architecture solide. Le choix des composants est aussi important que les algorithmes qui les feront fonctionner.

Prochaine séance :
Installer STM32CubeIDE et commencer la prise en main de la STM32 avec le premier programme GPIO.

Principe du projet

Aujourd'hui, je n'ai encore écrit aucune ligne de code.
Pourtant, le projet est déjà né.
Chaque grande réalisation commence par une décision.
Aujourd'hui, j'ai décidé de construire quelque chose qui me fera progresser.

Pourvu qu'aujourd'hui soit mieux qu'hier.

Séance 1 – GPIO

Journal de séance

Date : Juillet 2026

Objectif :
Faire clignoter la LED intégrée de la carte STM32 Nucleo-F446RE afin de valider la première interaction réelle avec le microcontrôleur.

Objectif pédagogique :
Comprendre le principe d'un GPIO non pas comme une simple fonction logicielle, mais comme un périphérique matériel contrôlé par des registres mémoire. L'objectif était aussi de commencer à comprendre la relation entre le processeur, la carte mémoire, les bus internes, les registres et les broches physiques du microcontrôleur.

Réalisation :

- Mise en place du dossier de travail de la séance 1.
- Création d'un projet STM32 minimal nommé `allume_led`.
- Mise en place d'un workflow de développement basé sur VS Code, CMake, Ninja, la toolchain `arm-none-eabi-gcc` et OpenOCD.
- Vérification de la détection du ST-Link intégré de la carte Nucleo-F446RE sous Linux.
- Compilation réussie du projet en mode `Debug`.
- Écriture d'un programme de clignotement de LED sans HAL, par accès direct aux registres.
- Flash du programme sur la STM32 avec OpenOCD.
- Validation expérimentale : la LED LD2 reliée à PA5 clignote correctement.
- Mise en place d'un fichier `.gitignore` propre pour éviter de versionner les fichiers générés.
- Commit Git et synchronisation du projet avec GitHub.

Choix technique :
Pour cette séance, le choix a été fait de ne pas utiliser HAL. L'objectif n'était pas seulement d'obtenir un résultat fonctionnel, mais de comprendre le fonctionnement interne du microcontrôleur. Le programme a donc été écrit en manipulant directement les adresses mémoire et les registres nécessaires.

Problèmes rencontrés :

- Le bouton *Launch STM32CubeMX* de l'extension VS Code ne trouvait pas STM32CubeMX, car STM32CubeMX n'était pas installé ou pas présent dans le chemin système.
- Le projet créé était un projet minimal et non un projet STM32Cube/HAL.
- La première configuration CMake échouait car Ninja n'était pas installé.
- La compilation échouait ensuite car la toolchain ARM `arm-none-eabi-gcc` n'était pas disponible dans le `PATH`.
- Certaines fonctions graphiques avancées de l'extension STM32 pour VS Code n'étaient pas directement disponibles car le projet n'était pas reconnu comme un projet STM32Cube complet.

Solutions trouvées :

— Utilisation du terminal comme workflow principal pour compiler et flasher le programme.

— Installation de Ninja, CMake et de la toolchain ARM.

— Compilation avec `cmake -preset Debug` puis `cmake -build -preset Debug`.

— Flash de la carte avec OpenOCD via l'interface ST-Link.

— Décision de conserver ce premier projet comme projet pédagogique minimal, centré sur la compréhension des registres et du fonctionnement matériel.

Notions comprises :

— Un GPIO est un périphérique matériel interne au microcontrôleur.

— Le processeur ne commande pas directement une LED : il écrit dans des registres mémoire.

— La STM32 utilise le principe du *Memory-Mapped I/O* : les périphériques sont accessibles à travers des adresses mémoire.

— `GPIOA_BASE` représente l'adresse de base du périphérique GPIOA.

— Les registres d'un périphérique sont situés à des offsets fixes à partir de son adresse de base.

— Le registre `MODER` configure le mode des broches GPIO.

— Le registre `ODR` permet de modifier l'état logique des sorties.

— Avant d'utiliser GPIOA, il faut activer son horloge via le registre `RCC_AHB1ENR`.

— AHB signifie *Advanced High-performance Bus*. C'est un bus interne rapide reliant le processeur à certains périphériques.

— Le mot-clé `volatile` est essentiel pour empêcher le compilateur d'optimiser les accès aux registres matériels.

— Les opérations binaires comme le décalage, le masquage et le XOR permettent de modifier précisément certains bits d'un registre.

Explication synthétique du fonctionnement :
Le processeur Cortex-M4 écrit dans le registre d'activation d'horloge du RCC afin d'activer le périphérique GPIOA. Ensuite, il configure la broche PA5 en sortie en modifiant les deux bits correspondants dans le registre `MODER`. Une fois PA5 configurée en sortie, le programme inverse régulièrement le bit 5 du registre `ODR`. Cette modification logique se traduit matériellement par un changement de tension sur la broche PA5, ce qui provoque l'allumage puis l'extinction de la LED LD2.

Méthode d'apprentissage retenue :
La méthode retenue pour la suite du projet est de comprendre chaque périphérique avant de l'utiliser. Pour chaque nouvelle brique, il faudra identifier le rôle du périphérique, sa position dans la carte mémoire, ses registres principaux, puis écrire le code correspondant. Le but n'est pas seulement d'utiliser une bibliothèque, mais d'apprendre à lire la documentation constructeur et à raisonner directement sur l'architecture du microcontrôleur.

Résultat :
La LED LD2 de la carte Nucleo-F446RE clignote correctement. La chaîne complète de développement est validée : édition du code, compilation, génération de l'exécutable, flash avec ST-Link et exécution sur la carte.

Apprentissage principal :

Le processeur ne commande jamais directement une LED. Il écrit dans des registres mémoire, et ce sont les périphériques matériels qui transforment ces écritures en actions physiques.

Prochaine étape :
Commencer la séance 2 avec l'UART afin de mettre en place un premier outil de debug série. Cette étape permettra d'afficher des messages depuis la STM32 vers le PC et de mieux observer l'état interne du programme pendant les essais.

Séance 2 – UART

Journal de séance

Date : Juillet 2026

Objectif :
Mettre en place une liaison série entre la STM32 Nucleo-F446RE et le PC afin de disposer d'une console de debug simple, fiable et réutilisable pendant tout le projet.

Objectif pédagogique :
Comprendre comment un périphérique UART est configuré en accès direct aux registres : activation des horloges, configuration d'une broche en fonction alternative, choix du débit, activation de l'émetteur et envoi des données caractère par caractère.

Configuration retenue :

- périphérique : `USART2` ;
- broche d'émission : `PA2` ;
- fonction alternative : `AF7` ;
- débit : 115200 bauds ;
- format : 8 bits de données, aucune parité, 1 bit de stop, soit `115200 8N1` ;
- terminal Linux : `picocom` sur `/dev/ttyACM0`.

Réalisation :

- Activation de l'horloge de `GPIOA` dans `RCC_AHB1ENR`.
- Activation de l'horloge de `USART2` dans `RCC_APB1ENR`.
- Configuration de `PA2` en mode fonction alternative dans `GPIOA_MODER`.
- Sélection de la fonction `AF7` dans `GPIOA_AFRL` afin de relier physiquement `PA2` à `USART2_TX`.
- Configuration du registre `USART2_BRR` avec la valeur `0x8B` pour obtenir environ 115200 bauds à partir d'une horloge de 16 MHz.
- Activation de l'émetteur avec le bit `TE` puis activation du périphérique avec le bit `UE` dans `USART2_CR1`.
- Écriture de la fonction `uart_send_char()` qui attend le bit `TXE` avant d'écrire dans le registre `USART2_DR`.
- Écriture de la fonction `uart_send_string()` pour transmettre une chaîne caractère par caractère jusqu'au caractère terminal `'\0'`.
- Écriture de la fonction `uart_send_uint()` pour convertir un entier non signé en caractères décimaux sans utiliser `printf()`.

— Utilisation d'une temporisation simple basée sur une boucle et l'instruction `nop` pour espacer les messages.

— Compilation avec CMake, flash avec OpenOCD et observation des messages dans `picocom`.

Résultats obtenus :
Le premier test a permis d'afficher correctement :

```
Hello Robot
```

Le programme a ensuite été enrichi afin d'afficher une suite de messages numérotés :

```
Message numero 1
Message numero 2
Message numero 3
```

La chaîne complète de communication a ainsi été validée :

STM32 → USART2 → PA2 → ST-Link → USB → `/dev/ttyACM0` → `picocom`

Organisation logicielle :
Le code UART a été séparé du programme principal afin de commencer à construire une architecture modulaire :

`Core/Inc/uart.h` : déclarations de l'interface publique ;
`Core/Src/uart.c` : configuration des registres et fonctions d'émission ;
`Core/Src/main.c` : logique principale de l'application.

Le fichier `main.c` ne manipule donc plus directement les registres de l'USART. Il utilise uniquement les fonctions `uart_init()`, `uart_send_char()`, `uart_send_string()` et `uart_send_uint()`.

Choix concernant `printf()` :
La redirection de `printf()` vers l'UART a volontairement été reportée. L'objectif de cette séance était de comprendre les mécanismes fondamentaux de transmission et la conversion manuelle des nombres, sans masquer le fonctionnement derrière la bibliothèque standard.

Problèmes rencontrés :

— Le terminal série `picocom` n'était pas initialement installé sur le système.

— Le message `Hello Robot` n'est envoyé qu'une fois au démarrage : si le terminal est ouvert après l'exécution du programme, le message n'est pas visible.

Solutions trouvées :

— Installation de `picocom` puis ouverture du port avec la commande `picocom -b 115200 /dev/ttyACM0`.

— Appui sur le bouton `RESET` de la carte après l'ouverture du terminal afin de relancer le programme et de recevoir le message initial.

Notions comprises :

— `TX` correspond à la transmission et `RX` à la réception.

- Une communication UART est asynchrone : les deux équipements doivent utiliser le même débit et le même format de trame.
- Une broche en fonction alternative est pilotée par un périphérique matériel et non directement par le registre `ODR` du GPIO.
- Le débit est défini par le registre `BRR` à partir de la fréquence d'horloge du périphérique.
- Le bit `TXE` indique que le registre de transmission peut recevoir un nouveau caractère.
- Le registre `DR` contient la donnée à transmettre.
- Une chaîne de caractères est un tableau terminé par `'\0'`.
- Un entier doit être converti en caractères avant de pouvoir être affiché dans un terminal.
- La séparation entre interface `.h`, implémentation `.c` et programme principal rend le code plus lisible et réutilisable.

Apprentissage principal :
L'UART transforme les données écrites par le processeur dans des registres en une suite de bits transmise sur une broche physique. Grâce à cette liaison, le programme embarqué peut rendre visible son état interne sur le PC.

Prochaine étape :
Utiliser cette console UART pendant la séance 3 pour afficher le résultat de la première communication I2C avec le MPU6050, notamment la valeur de son registre d'identification `WHO_AM_I`.

Séance 3 – IMU : première communication

Journal de séance

Date : Juillet 2026

Objectif :
Établir une première communication I2C entre la STM32 Nucleo-F446RE et le module MPU6050, puis vérifier l'identité du capteur en lisant son registre `WHO_AM_I`.

Objectif pédagogique :
Comprendre le chemin complet d'une communication avec un capteur numérique : câblage électrique, configuration des broches en fonction alternative, activation du périphérique I2C, réglage des temporisations, émission d'une adresse, gestion des acquittements et lecture d'un registre interne.

Configuration retenue :

- périphérique STM32 : `I2C1` ;
- ligne d'horloge : `PB8 / I2C1_SCL` ;
- ligne de données : `PB9 / I2C1_SDA` ;
- fonction alternative : `AF4` ;
- mode électrique : sorties *open-drain* avec résistances de tirage ;
- fréquence de l'horloge APB1 : 16 MHz ;
- fréquence du bus I2C : 100 kHz ;
- adresse I2C du MPU6050 : `0x68` ;

— registre d'identification : `WHO_AM_I` à l'adresse interne `0x75` ;
— valeur attendue : `0x68`, soit 104 en décimal.

Câblage réalisé :

Nucleo-F446RE	Broche STM32	MPU6050
3,3 V	Alimentation	VCC
GND	Masse commune	GND
D15	PB8 / I2C1_SCL	SCL
D14	PB9 / I2C1_SDA	SDA

Le module s'est correctement allumé après le câblage. Cette LED confirme la présence de l'alimentation, mais elle ne suffit pas à prouver que les échanges I2C fonctionnent.

Réalisation :

— Configuration de `PB8` et `PB9` en mode fonction alternative dans `GPIOB_MODER`.
— Sélection de la fonction `AF4` dans `GPIOB_AFRH` afin de relier les broches au périphérique `I2C1`.
— Configuration des deux lignes en sorties *open-drain*, avec vitesse élevée et résistances de tirage internes.
— Activation de l'horloge de `GPIOB` dans `RCC_AHB1ENR`.
— Activation de l'horloge de `I2C1` dans `RCC_APB1ENR` avec le bit `I2C1EN`.
— Configuration du champ `FREQ` de `I2C1_CR2` avec la valeur 16, correspondant à une horloge APB1 de 16 MHz.
— Configuration du registre `CCR` avec la valeur 80 afin d'obtenir un bus I2C à 100 kHz en mode standard.
— Configuration du registre `TRISE` avec la valeur 17 pour tenir compte du temps de montée maximal des lignes en mode standard.
— Activation du périphérique grâce au bit `PE` du registre `I2C1_CR1`.
— Écriture d'une fonction `i2c1_probe()` générant une condition `START`, envoyant l'adresse `0x68` et vérifiant la réception d'un acquittement `ACK`.
— Écriture d'une fonction `i2c1_read_register()` permettant de sélectionner un registre interne, de produire un *Repeated START*, puis de recevoir un octet.
— Lecture du registre `WHO_AM_I` et affichage de sa valeur dans `picocom` grâce au pilote UART de la séance 2.

Calcul des temporisations :
Le champ `FREQ` reçoit la fréquence APB1 exprimée en MHz :

$$\texttt{CR2.FREQ} = 16$$

En mode standard, la valeur de `CCR` est calculée par :

$$\texttt{CCR} = 16\,000\,000 / (2 \times 100\,000) = 80$$

Pour le temps de montée maximal de 1 µs, une horloge de 16 MHz représente 16 cycles de 62,5 ns. Le registre est donc programmé avec :

$$\texttt{TRISE} = 16 + 1 = 17$$

Séquence de lecture de `WHO_AM_I` :

`START` → `0x68 + écriture` → `ACK` → `registre 0x75` → *Repeated START* → `0x68 + lecture` → `ACK` → `donnée reçue` → `NACK + STOP`

La première adresse, `0x68`, sélectionne le composant présent sur le bus. L'adresse `0x75` sélectionne ensuite un registre précis à l'intérieur de ce composant. Ces deux adresses ne jouent donc pas le même rôle.

Résultats obtenus :
Le premier test d'adressage a produit :

```
Test de l'adresse I2C 0x68...
MPU6050 detecte a l'adresse 0x68
```

La lecture du registre d'identification a ensuite produit :

```
Lecture du registre WHO_AM_I...
   WHO_AM_I = 104 en decimal
MPU6050 correctement identifie
```

La valeur 104 en décimal correspond à `0x68` en hexadécimal. Elle confirme que le composant répond à l'adresse prévue et renvoie l'identifiant attendu.

Problèmes rencontrés :

- Lors de la première compilation, les fonctions `uart_init()` et `uart_send_string()` étaient déclarées mais introuvables pendant l'édition des liens.
- Le fichier `uart.c` était présent dans le dossier `Src`, mais il n'était pas ajouté explicitement aux sources du projet dans `CMakeLists.txt`.
- Le registre `TRISE` affichait initialement la valeur 2 au lieu de 17, car la fonction de configuration n'était pas effectivement appelée avant la lecture du registre.
- Une opération automatique de refactorisation proposée par VS Code tentait d'ajouter le fichier d'en-tête `i2c.h` de manière inadaptée dans `add_executable()`.

Solutions trouvées :

- Ajout explicite de `Src/uart.c` puis de `Src/i2c.c` dans `target_sources()` du fichier `CMakeLists.txt`.
- Reconfiguration et recompilation propre du projet avec CMake et Ninja.
- Vérification directe de la valeur de `TRISE` avant et après l'écriture, ce qui a permis d'identifier l'appel manquant.
- Annulation de la proposition de refactorisation automatique et modification manuelle du fichier CMake.

Organisation logicielle :
Après validation du prototype dans `main.c`, le pilote I2C a été séparé afin de conserver une architecture claire :

Fichier	Rôle
`Inc/i2c.h`	Interface publique : initialisation, test d'adresse et lecture d'un registre.
`Src/i2c.c`	Registres RCC/GPIO/I2C, fonctions internes de configuration et transactions I2C.
`Src/main.c`	Initialisation générale, test du MPU6050 et messages de diagnostic UART.

Les fonctions internes de configuration sont déclarées `static`, car elles appartiennent uniquement au pilote. Les fonctions `i2c1_init()`, `i2c1_probe()` et `i2c1_read_register()` constituent l'interface publique utilisée par le programme principal.

Notions comprises :

- La STM32 joue le rôle de maître et génère l'horloge sur `SCL`.
- `SDA` est une ligne bidirectionnelle utilisée pour les adresses, les données et les acquittements.
- Une sortie *open-drain* peut forcer une ligne à zéro ou la relâcher ; la remontée à l'état haut est assurée par une résistance de tirage.
- L'adresse I2C du périphérique et l'adresse d'un registre interne sont deux informations différentes.
- L'adresse I2C `0x68` est codée sur 7 bits ; le contrôleur ajoute ensuite le bit lecture/écriture dans l'octet transmis.
- Un `ACK` confirme que le destinataire a reconnu l'adresse ou reçu un octet.
- Le bit `AF` signale au contraire un défaut d'acquittement.
- La lecture d'un registre utilise généralement une phase d'écriture pour sélectionner le registre, puis une phase de lecture introduite par un *Repeated START*.
- Les bits d'état `SB`, `ADDR`, `TXE`, `BTF` et `RXNE` permettent de suivre l'avancement matériel de la transaction.
- Des temporisations logicielles évitent de bloquer indéfiniment le processeur lorsqu'un événement I2C ne se produit pas.

Résultat final :
La chaîne complète est validée : la STM32 configure I2C1, contacte le MPU6050 à l'adresse `0x68`, sélectionne le registre `WHO_AM_I`, reçoit la valeur `0x68` et l'affiche sur le terminal série.

Cortex-M4 → RCC → GPIOB/AF4 → I2C1 → PB8/PB9 → MPU6050 → UART → PC

Apprentissage principal :
Avant d'utiliser les mesures d'un capteur dans un algorithme, il faut d'abord valider séparément son alimentation, son câblage, son adresse, ses transactions et son identité. Une communication correctement diagnostiquée devient une base fiable pour les étapes suivantes.

Prochaine étape :
Commencer la séance 4 avec la génération d'un signal PWM à l'aide d'un timer STM32. La lecture des données brutes de l'accéléromètre et du gyroscope sera abordée plus tard, pendant la séance 7.

Séance 4 – PWM moteur

Journal de séance

Date : Août 2026

Objectif :
Générer un signal PWM matériel avec un timer de la STM32 Nucleo-F446RE, faire varier

son rapport cyclique de 0 à 100 % et valider expérimentalement la tension moyenne obtenue sur une broche.

Objectif pédagogique :
Comprendre comment un timer transforme une horloge interne en un signal périodique : division de fréquence avec le prescaler, comptage jusqu'à une valeur maximale, comparaison avec un registre de capture/compare et routage du signal vers une broche configurée en fonction alternative.

Configuration retenue :

- timer : `TIM3` ;
- canal PWM : `TIM3_CH1` ;
- broche de sortie : `PA6` ;
- fonction alternative : `AF2` ;
- horloge du timer considérée : 16 MHz ;
- prescaler : `PSC = 15` ;
- auto-reload : `ARR = 999` ;
- fréquence PWM obtenue : 1 kHz ;
- rapport cyclique réglable avec `CCR1` ;
- mode de sortie : PWM mode 1.

Réalisation :

- Activation de l'horloge de `GPIOA` dans `RCC_AHB1ENR`.
- Activation de l'horloge de `TIM3` dans `RCC_APB1ENR`.
- Configuration de `PA6` en mode fonction alternative dans `GPIOA_MODER`.
- Sélection de `AF2` dans `GPIOA_AFRL` afin de relier `PA6` à `TIM3_CH1`.
- Configuration de la sortie en *push-pull*, sans résistance de tirage.
- Arrêt du timer pendant sa configuration avec le bit `CEN`.
- Programmation de `PSC = 15` afin de diviser 16 MHz par 16 et d'obtenir un compteur à 1 MHz.
- Programmation de `ARR = 999` afin d'obtenir une période de 1000 coups de compteur, soit un signal à 1 kHz.
- Configuration du canal 1 en PWM mode 1 avec le champ `OC1M = 110`.
- Activation du preload de `CCR1` avec `OC1PE` et du preload de `ARR` avec `ARPE`.
- Activation de la sortie du canal avec le bit `CC1E` du registre `CCER`.
- Génération d'un événement de mise à jour avec le bit `UG` afin de charger immédiatement `PSC`, `ARR` et `CCR1`.
- Création de la fonction `pwm_set_duty()` pour convertir un pourcentage compris entre 0 et 100 en une valeur de `CCR1`.
- Séparation du pilote dans `Inc/pwm.h` et `Src/pwm.c`.
- Affichage des informations de test avec le pilote UART bare-metal, sans utiliser `printf()`.
- Réalisation d'un test automatique faisant évoluer le rapport cyclique de 0 à 100 %, puis de 100 à 0 %, par pas de 10 %.

Calcul de la fréquence PWM :
La fréquence du compteur est donnée par :

```
f_compteur = 16 000 000 / (15 + 1) = 1 000 000 Hz
```

Le compteur parcourt ensuite les valeurs de 0 à 999, soit 1000 états :

```
f_PWM = 1 000 000 / (999 + 1) = 1 000 Hz
```

La période du signal vaut donc 1 ms.

Calcul du rapport cyclique :

```
rapport cyclique = CCR1 / (ARR + 1)
```

Pour 60 % :

```
CCR1 = (999 + 1) × 60 / 100 = 600
```

En PWM mode 1, la sortie reste active lorsque `CNT < CCR1`, puis devient inactive lorsque `CNT >= CCR1`.

Organisation logicielle :

Fichier	Rôle
`Inc/pwm.h`	Interface publique : initialisation du PWM et réglage du rapport cyclique.
`Src/pwm.c`	Adresses mémoire, registres GPIO/RCC/TIM3 et implémentation du pilote.
`Src/main.c`	Initialisation générale, test automatique et messages de diagnostic UART.

Le programme principal utilise uniquement `pwm_init()` et `pwm_set_duty()`. Il ne manipule plus directement les registres de `TIM3`.

Résultats affichés dans `picocom` :

```
Demarrage du robot
Initialisation du PWM...
PWM TIM3_CH1 actif sur PA6
Frequence : 1000 Hz
Rapport cyclique : 60 %
```

Validation au multimètre :
Le multimètre a été placé entre `PA6` et la masse de la carte. Il mesure la valeur moyenne du signal PWM.

Rapport cyclique	Valeur théorique	Valeur mesurée
0 %	0 V	0 V
25 %	0,825 V	0,826 V
60 %	1,980 V	1,983 V
100 %	environ 3,3 V	3,306 V

Les valeurs mesurées sont très proches des valeurs théoriques. Elles confirment que la fonction `pwm_set_duty()` modifie correctement `CCR1` et que le signal est bien routé vers `PA6`.

Test automatique :

$$0\ \% \rightarrow 10\ \% \rightarrow 20\ \% \rightarrow \cdots \rightarrow 100\ \% \rightarrow 90\ \% \rightarrow \cdots \rightarrow 0\ \%$$

Une temporisation approximative maintient chaque valeur suffisamment longtemps pour observer la montée puis la descente de la tension moyenne au multimètre.

Point de vigilance concernant la boucle descendante :
La variable de boucle descendante est signée. Une variable non signée ne peut pas devenir négative : après zéro, une soustraction provoquerait un débordement et pourrait rendre la boucle infinie.

Notions comprises :

- Le PWM est un signal numérique périodique, et non une véritable tension analogique variable.
- Le prescaler `PSC` règle la vitesse du compteur.
- Le registre `ARR` fixe la période du signal.
- Le registre `CCR1` fixe la durée de l'état actif et donc le rapport cyclique.
- Le compteur parcourt les valeurs de 0 à `ARR`, ce qui représente `ARR + 1` états.
- En PWM mode 1, la sortie est active tant que `CNT < CCR1`.
- Les bits de preload permettent d'appliquer les nouvelles valeurs proprement au début d'une période.
- Une fonction alternative relie la sortie matérielle d'un timer à une broche physique.
- Le multimètre affiche la tension moyenne du PWM, mais il ne montre ni les fronts ni la fréquence exacte du signal.
- La STM32 ne devra pas alimenter directement un moteur : le signal PWM commandera plus tard le driver de puissance TB6612FNG.

Résultat final :

Cortex-M4 → RCC → TIM3 → comparaison `CNT/CCR1` → `TIM3_CH1` → AF2 → PA6

Le pilote génère un signal à 1 kHz dont le rapport cyclique peut être réglé de 0 à 100 %. Les mesures expérimentales obtenues sont conformes aux calculs.

Apprentissage principal :
Un timer matériel permet de produire un signal précis sans demander au processeur de basculer continuellement une broche. Le processeur configure la fréquence et le rapport cyclique, puis le périphérique timer génère lui-même le signal.

Limite volontaire de la séance :
Le driver TB6612FNG et les moteurs n'ont pas été connectés pendant cette séance. Leur commande par PWM et GPIO de direction sera traitée pendant la séance 6.

Prochaine étape :
Commencer la séance 5 avec les encodeurs afin de mesurer le mouvement réel des moteurs avant l'intégration complète du driver de puissance.

Séance 5 – Encodeurs

Journal de séance

Date : Août 2026

Objectif :
Mesurer le mouvement réel d'un motoréducteur à l'aide de son encodeur quadrature, déterminer son sens de rotation, convertir le comptage en position angulaire puis calculer sa vitesse réelle en tours par minute.

Objectif pédagogique :
Comprendre qu'un timer STM32 peut être utilisé autrement que comme base de temps ou générateur PWM. En mode encodeur, le compteur du timer n'est plus principalement cadencé par le temps : il évolue en fonction des transitions reçues sur les deux voies A et B. La séance vise aussi à distinguer clairement mesure de position, mesure de temps et calcul de vitesse.

Configuration retenue :

- encodeur quadrature : voies A et B du motoréducteur ;
- timer encodeur : `TIM2` ;
- voie A : `PA0 / TIM2_CH1` ;
- voie B : `PA1 / TIM2_CH2` ;
- fonction alternative : `AF1` ;
- mode du timer : Encoder mode 3, avec `SMS = 011` ;
- compteur : `TIM2_CNT`, utilisé comme compteur signé côté logiciel ;
- plage : `ARR = 0xFFFFFFFF` ;
- base de temps indépendante : `TIM5` ;
- horloge considérée pour TIM5 : 16 MHz ;
- `TIM5_PSC = 15`, soit un compteur à 1 MHz et un tick de 1 µs ;
- période d'échantillonnage de la vitesse : environ 100 ms ;
- calibration retenue : **3881 comptes par tour de l'axe de sortie**.

Principe de l'encodeur quadrature :
Les deux sorties A et B sont des signaux numériques déphasés. L'ordre dans lequel leurs états changent permet de déterminer le sens de rotation. Dans un sens, la séquence des états peut être vue comme :

$$\texttt{00} \rightarrow \texttt{10} \rightarrow \texttt{11} \rightarrow \texttt{01} \rightarrow \texttt{00}$$

et dans l'autre sens comme la séquence inverse :

$$\texttt{00} \rightarrow \texttt{01} \rightarrow \texttt{11} \rightarrow \texttt{10} \rightarrow \texttt{00}$$

Le timer observe matériellement les transitions de `TI1` et `TI2`. Selon leur ordre, il incrémente ou décrémente automatiquement `TIM2_CNT`. Le programme C n'exécute donc pas lui-même `compteur++` ou `compteur-` à chaque front.

Chemin matériel validé :

Encodeur A/B → PA0/PA1 → AF1 → TIM2_CH1/TIM2_CH2 → TI1/TI2 → logique quadrature → `TIM2_CNT`

Configuration de TIM2 :

- Activation de l'horloge de `GPIOA` dans `RCC_AHB1ENR`.
- Configuration de `PA0` et `PA1` en fonction alternative dans `GPIOA_MODER`.
- Sélection de `AF1` dans `GPIOA_AFRL` afin de relier PA0 à `TIM2_CH1` et PA1 à `TIM2_CH2`.
- Activation de l'horloge de `TIM2` avec `TIM2EN` dans `RCC_APB1ENR`.
- Configuration de `CC1S = 01` et `CC2S = 01` dans `TIM2_CCMR1` afin de connecter les entrées `IC1` et `IC2` à `TI1` et `TI2`.
- Polarité normale des deux entrées avec `CC1P = 0` et `CC2P = 0` dans `TIM2_CCER`.
- Sélection du mode encodeur 3 avec `SMS = 011` dans `TIM2_SMCR`.
- Utilisation de toute la plage 32 bits avec `TIM2_ARR = 0xFFFFFFFF`.
- Initialisation de `TIM2_CNT` à zéro.
- Démarrage du compteur avec le bit `CEN` de `TIM2_CR1`.

Organisation logicielle :

Fichier	Rôle
`Inc/encoder.h`	Interface publique : initialisation, lecture du compteur et calcul de vitesse.
`Src/encoder.c`	Registres GPIO/RCC/TIM2, configuration du mode encodeur et conversions liées au mouvement.
`Inc/timebase.h`	Interface de la base de temps en microsecondes.
`Src/timebase.c`	Configuration de TIM5 comme chronomètre matériel à 1 µs par tick.
`Src/main.c`	Échantillonnage, calcul de ΔCNT, calcul de Δt et affichage UART.

Premier test de position :
Le moteur n'a pas été alimenté pendant le premier essai. L'axe a été tourné à la main afin de vérifier uniquement l'encodeur. Le compteur est resté stable lorsque l'axe était immobile, a augmenté dans un sens et diminué dans l'autre.
Extraits représentatifs :

```
Position : 336
Position : 336
Position : 691
Position : 520
Position : 202
Position : -14
Position : -241
Position : -746
```

Ces essais valident simultanément la réception des deux voies, le comptage matériel et la détection du sens de rotation.

Calibration expérimentale :
Pour convertir le compteur en grandeur physique, plusieurs mesures de dix tours complets de l'axe de sortie ont été réalisées :

Essai	Comptes pour 10 tours	Comptes par tour
1	38826	3882,6
2	38809	3880,9
3	38809	3880,9

La moyenne expérimentale vaut environ 3881,47 comptes par tour. Pour garder une constante simple dans le code, la valeur retenue est :

3881 comptes/tour

Cette calibration est utilisée pour toutes les conversions suivantes.

Conversion en angle :
Un tour complet correspond à 360 degrés. L'angle cumulé de l'axe peut donc être estimé par :

$$\theta = \frac{CNT}{3881} \times 360^\circ$$

Ainsi, environ 1940 comptes correspondent à un demi-tour, soit environ 180 degrés. Le signe du compteur indique le sens de rotation.

Création d'une vraie base de temps :
Une temporisation logicielle approximative n'est pas adaptée au calcul d'une vitesse. Un deuxième timer, `TIM5`, a donc été configuré comme chronomètre matériel indépendant de TIM2.
Avec une horloge de 16 MHz et `PSC = 15` :

```
f_CNT = 16 000 000 / (15 + 1) = 1 000 000 Hz
```

Chaque incrément de `TIM5_CNT` représente alors 1 µs. Le registre `EGR` est utilisé avec le bit `UG` afin de charger immédiatement la valeur du prescaler avant le démarrage du compteur.
Le test de la base de temps a montré une augmentation continue de `TIM5_CNT`, validant le fonctionnement du chronomètre matériel.

Calcul de la vitesse :
Deux positions sont lues à deux instants différents :

$$\Delta CNT = CNT_{actuel} - CNT_{precedent}$$

$$\Delta t = t_{actuel} - t_{precedent}$$

La vitesse en tours par minute est ensuite calculée avec :

$$RPM = \frac{\Delta CNT \times 60 \times 1\,000\,000}{3881 \times \Delta t_{\mu s}}$$

Le calcul est effectué environ toutes les 100 ms. La condition utilise `delta_us >= 100000` et non une égalité stricte, car le processeur n'est pas garanti de lire exactement la valeur 100000.

Choix d'un calcul entier :
Une première version du calcul utilisait des nombres flottants. Pendant les essais bare-metal, l'exécution s'arrêtait précisément au moment du calcul de la vitesse en `float`. Afin de conserver cette séance simple et indépendante d'une configuration supplémentaire de la FPU, le calcul a été réécrit en arithmétique entière.
Le numérateur est calculé en `int64_t` afin d'éviter un débordement lors du produit :

`delta_count` × 60 × 1 000 000

Le résultat final est retourné en `int32_t`. Les décimales sont volontairement perdues, ce qui est suffisant pour la validation de cette séance. La configuration et l'utilisation de la FPU seront traitées lorsque les calculs flottants deviendront réellement nécessaires, notamment pour le filtrage et le contrôle.

Résultats de vitesse :
Les essais à la main ont produit des valeurs cohérentes dans les deux sens. Lorsque l'axe est immobile, la variation vaut zéro et la vitesse affichée est nulle. En rotation, le signe de la vitesse suit le signe de ΔCNT.

```
  Position : 43932 | Delta : 0 | Vitesse : 0 tr/min
Position : 42581 | Delta : -249 | Vitesse : -38 tr/min
Position : 40824 | Delta : -363 | Vitesse : -56 tr/min
 Position : 39076 | Delta : 171 | Vitesse : 26 tr/min
 Position : 39700 | Delta : 213 | Vitesse : 32 tr/min
  Position : 38812 | Delta : 0 | Vitesse : 0 tr/min
```

Ces mesures montrent correctement l'arrêt, l'accélération, le ralentissement et l'inversion du sens de rotation.

Conversion future en distance et en vitesse linéaire :
Si une roue de diamètre D est montée sur l'axe, sa circonférence vaut πD. En l'absence de glissement :

$$d = \frac{CNT}{3881} \times \pi D$$

et, si D est exprimé en mètres, la vitesse linéaire vaut :

$$v = \frac{\pi D \times RPM}{60}$$

Cette relation constituera plus tard la base de l'odométrie du robot.

Problèmes rencontrés :

- Confusion initiale sur le rôle du timer en mode encodeur : il fallait le voir ici comme un compteur matériel piloté par les fronts A/B et non comme un simple découpage du temps.
- Le premier programme de vitesse affichait les messages d'initialisation puis plus rien.
- Le test isolé de TIM5 a confirmé que la base de temps fonctionnait correctement.
- Des traces UART placées étape par étape ont montré que le blocage apparaissait après le calcul de ΔCNT, au moment du calcul flottant des RPM.

Solutions trouvées :

- Décomposition du problème en tests séparés : TIM2 pour le mouvement, TIM5 pour le temps, puis combinaison des deux.
- Instrumentation du programme avec des messages UART afin de localiser précisément le point de blocage.
- Remplacement du calcul `float` par un calcul entier utilisant `int64_t` pour les produits intermédiaires.
- Conservation d'une période réelle mesurée avec TIM5 au lieu de supposer que la boucle logicielle dure exactement 100 ms.

Notions comprises :

- Un encodeur quadrature utilise deux voies déphasées afin de mesurer le déplacement et son sens.
- Les fronts montants et descendants constituent les événements qui font évoluer le compteur en mode encodeur.
- Un même périphérique timer peut servir à des fonctions très différentes selon sa configuration : PWM, base de temps ou interface encodeur.
- `CCMR1` choisit comment les canaux sont utilisés, `SMCR` sélectionne le mode encodeur, `CCER` règle la polarité et `CR1.CEN` démarre le compteur.
- Le processeur configure le timer, mais le comptage des fronts est ensuite effectué matériellement.
- Une position est une valeur cumulée de compteur, alors qu'une vitesse dépend de la variation de cette position pendant un intervalle de temps.
- Une calibration expérimentale sur plusieurs tours réduit l'erreur liée au positionnement manuel.
- Le signe de ΔCNT permet de conserver l'information de sens dans la vitesse.
- Une vraie base de temps matérielle est nécessaire pour obtenir une mesure de vitesse fiable.
- L'arithmétique entière 64 bits permet de réaliser des calculs précis sans dépendre immédiatement de la FPU.

Résultat final :

Rotation mécanique → Encodeur A/B → TIM2 → `CNT` → position / angle

`CNT` + TIM5 → $\Delta CNT/\Delta t$ → vitesse en tr/min

La STM32 mesure désormais automatiquement la position relative et le sens de rotation du moteur, puis calcule sa vitesse réelle à partir d'une base de temps matérielle indépendante.

Apprentissage principal :
Le PWM de la séance précédente permettait de commander une action sans savoir exactement ce que le moteur faisait. L'encodeur ferme la chaîne de mesure : il transforme le mouvement mécanique réel en information numérique exploitable par le microcontrôleur.

Prochaine étape :
Commencer la séance 6 avec le driver TB6612FNG afin de commander réellement le moteur en avant et en arrière avec GPIO + PWM, tout en conservant l'encodeur comme retour de mesure.

Séance 6 – Lecture moteur + sens de rotation

Journal de séance

Date : Août 2026

Objectif :
Commander réellement un motoréducteur avec le driver TB6612FNG, contrôler son sens de rotation et sa puissance avec GPIO + PWM, puis vérifier la cohérence de la commande à l'aide de l'encodeur.

Objectif pédagogique :
Comprendre le rôle d'un pont en H comme interface entre la logique 3,3 V de la STM32 et la puissance nécessaire au moteur. La séance vise aussi à séparer clairement trois fonctions : l'activation du driver avec `STBY`, le choix du sens avec `AIN1/AIN2` et le réglage de la commande moteur avec `PWMA`. Enfin, les mesures de la séance 5 sont réutilisées afin de comparer la commande PWM au mouvement réellement observé.

Configuration retenue :

- driver moteur : `TB6612FNG`, canal A ;
- `PC0` → `AIN1` ;
- `PC1` → `AIN2` ;
- `PC2` → `STBY` ;
- `PA6 / TIM3_CH1` → `PWMA` ;
- `AO1/AO2` → les deux fils de puissance du moteur ;
- logique du TB6612FNG : 3,3 V depuis la STM32 ;
- alimentation moteur : environ 5 V obtenus avec le convertisseur Buck LM2596 ;
- masse commune entre STM32, TB6612FNG, Buck et encodeur ;
- encodeur conservé sur `PA0 / TIM2_CH1` et `PA1 / TIM2_CH2` ;
- TIM5 conservé comme base de temps pour le calcul de vitesse.

Préparation de l'alimentation :
La batterie LiPo 2S ne doit pas être utilisée directement comme tension de test du moteur. Le convertisseur Buck LM2596 a donc été placé entre la batterie et l'entrée `VM` du TB6612FNG. La tension de sortie a été réglée au multimètre à environ 5,0 V avant de connecter le driver. Le potentiomètre du module étant multi-tours, plusieurs rotations ont été nécessaires avant d'observer une variation visible de la tension. Cette étape a montré l'importance de toujours régler et vérifier la sortie d'un convertisseur avant de l'appliquer au circuit de puissance.

LiPo 2S → Buck LM2596 réglé à 5 V → `VM` du TB6612FNG → `AO1/AO2` → moteur

La logique reste alimentée séparément :

`STM32 3,3 V` → `VCC du TB6612FNG`

Les masses de la STM32, du driver et de l'alimentation moteur sont reliées afin que tous les niveaux logiques utilisent la même référence électrique.

Principe de commande du canal A :
Le sens est déterminé par les entrées `AIN1` et `AIN2`, tandis que `PWMA` reçoit le PWM produit pendant la séance 4. La broche `STBY` doit rester à l'état haut pour autoriser le fonctionnement du driver.

`AIN1`	`AIN2`	Utilisation dans le projet
1	0	Rotation dans le sens appelé `AVANT`.
0	1	Rotation dans le sens appelé `ARRIERE`.
0	0	État utilisé lors de l'arrêt logiciel.

Le rapport cyclique de `PWMA` ne choisit donc pas le sens : il règle la commande appliquée au moteur. Les GPIO `PC0`, `PC1` et `PC2` sont configurés en sorties *push-pull*, car la STM32 doit imposer directement des niveaux logiques bas ou hauts au driver.

Organisation logicielle :
Un pilote moteur séparé a été créé afin de ne pas mélanger la logique de direction avec la génération PWM :

Fichier	Rôle
`Inc/moteur.h`	Interface publique du pilote moteur.
`Src/moteur.c`	Configuration de GPIOC et commande de `AIN1`, `AIN2` et `STBY`.
`Inc/pwm.h / Src/pwm.c`	Génération du PWM sur `PA6 / TIM3_CH1`.
`Inc/encoder.h / Src/encoder.c`	Lecture de la position et calcul de la vitesse avec TIM2.
`Inc/timebase.h / Src/timebase.c`	Base de temps TIM5 utilisée pour les mesures.
`Src/main.c`	Séquences de test, affichage UART et comparaison PWM / vitesse.

Les principales fonctions ajoutées sont :

- `moteur_init()` ;
- `moteur_avant()` ;
- `moteur_arriere()` ;
- `moteur_arreter()` ;
- `moteur_activer()` ;
- `moteur_desactiver()`.

La séparation retenue est donc :

`moteur.c` → direction + activation `pwm.c` → commande PWM

Le programme principal peut ainsi demander un sens avec `moteur_avant()` ou `moteur_arriere()`, puis régler le rapport cyclique avec `pwm_set_duty()` sans manipuler directement les registres.

Premier test fonctionnel :
Un premier essai a été réalisé avec une commande de 30 %. Le moteur a démarré correctement, validant la chaîne complète :

STM32 → GPIO de direction + PWM → TB6612FNG → moteur

Une séquence automatique `AVANT` → `ARRET` → `ARRIERE` → `ARRET` a ensuite confirmé l'inversion correcte du pont en H.

Test de variation du PWM :
Le moteur a ensuite été testé successivement à 20 %, 40 %, 60 % et 80 %. À l'observation, sa vitesse augmente progressivement lorsque le rapport cyclique augmente.
L'encodeur de la séance 5 a ensuite été réintégré afin de mesurer cette évolution. Pour le premier moteur, les régimes établis représentatifs obtenus sont :

PWM	ΔCNT sur environ 100 ms	Vitesse stabilisée
20 %	environ -133 à -134	environ -20 tr/min
40 %	environ -306 à -308	environ -47 tr/min
60 %	environ -471 à -475	environ -72 à -73 tr/min
80 %	environ -621 à -628	environ -96 à -97 tr/min

Ces résultats montrent directement qu'une augmentation du rapport cyclique produit une augmentation de la valeur absolue de la vitesse mesurée. Ils montrent aussi que le rapport cyclique représente une commande électrique et non un pourcentage exact de la vitesse maximale.

Observation de la réponse dynamique :
Les premières mesures de chaque essai montrent que le moteur n'atteint pas instantanément son régime établi. À 60 %, par exemple, la vitesse a évolué progressivement d'environ -22 tr/min vers -70 tr/min. À 80 %, elle a progressé d'environ -27 tr/min vers -96 tr/min.
Cette montée progressive met en évidence l'inertie mécanique et le temps nécessaire au moteur pour atteindre sa vitesse de régime.

Validation du sens avec l'encodeur :
Un essai complémentaire a été réalisé à PWM constant de 60 % en inversant seulement `AIN1` et `AIN2`.
Pour le moteur 1 :

Commande	Signe de ΔCNT	Vitesse stabilisée
AVANT	négatif	environ -70 tr/min
ARRIERE	positif	environ +69 tr/min

Le moteur 2 a ensuite été testé de la même manière :

Commande	Signe de ΔCNT	Vitesse stabilisée
AVANT	négatif	environ -67 à -68 tr/min
ARRIERE	positif	environ +66 à +67 tr/min

Dans le câblage actuel, la convention `AVANT` correspond donc à une vitesse encodeur négative et `ARRIERE` à une vitesse positive. Cette convention est cohérente sur les deux moteurs et peut être conservée ou corrigée plus tard au niveau logiciel selon la convention choisie pour le robot.

Vérification du calcul de vitesse :
À 60 %, une valeur typique du moteur 1 est d'environ $\Delta CNT = -458$ sur 0,1 s. Avec 3881 comptes par tour :

$$RPM \approx \frac{-458 \times 60}{3881 \times 0{,}1} \approx -70{,}8 \text{ tr/min}$$

La valeur calculée est cohérente avec l'affichage UART, proche de -70 tr/min. La chaîne encodeur + TIM2 + TIM5 + calcul entier reste donc valide lorsque le moteur est réellement piloté par le TB6612FNG.

Problèmes et observations rencontrés :

— Le réglage du Buck LM2596 semblait initialement ne pas réagir ; son potentiomètre multi-tours nécessitait simplement plusieurs rotations avant de faire varier sensiblement la tension.

— Le signe de la vitesse est négatif pour la commande appelée `AVANT`. Ce comportement n'est pas une erreur : il dépend de l'ordre des voies A/B et de la convention choisie.

— Après la suppression de la commande, la position encodeur peut encore évoluer pendant un court instant : le moteur possède une inertie et ne s'immobilise pas instantanément.

— Les deux moteurs ne donnent pas exactement la même vitesse pour un même rapport cyclique, ce qui est normal pour des actionneurs réels et sera traité plus tard lors des boucles de contrôle.

Notions comprises :

— La STM32 fournit les signaux de commande, tandis que le TB6612FNG commute l'énergie provenant de l'alimentation moteur.

— `VCC` alimente la logique du driver alors que `VM` alimente l'étage de puissance moteur.

— Une masse commune est nécessaire pour donner une référence électrique commune aux signaux de commande.

— `STBY` active ou désactive le driver.

— `AIN1/AIN2` déterminent le sens de rotation.

— `PWMA` reçoit le signal PWM et règle la commande appliquée au moteur.

— Les broches de direction sont utilisées en *push-pull*, contrairement aux lignes I2C configurées en *open-drain*.

— Le PWM demandé et la vitesse réelle sont deux grandeurs différentes.

— L'encodeur permet d'observer la réponse réelle du moteur et de conserver l'information de sens grâce au signe de ΔCNT.

— La vitesse augmente progressivement au démarrage : le moteur possède une dynamique et une inertie.

— Deux moteurs identiques peuvent présenter de petites différences de vitesse pour une même commande.

Résultat final :

Commande : STM32 → `PC0/PC1/PC2` + `PA6/TIM3` → TB6612FNG → moteur

Mesure : moteur → encodeur A/B → TIM2 → ΔCNT + TIM5 → Δt → vitesse

La STM32 sait désormais commander le sens et le niveau de commande d'un moteur à travers le TB6612FNG tout en mesurant simultanément le mouvement réellement obtenu avec l'encodeur.

Apprentissage principal :
Le PWM indique ce que le microcontrôleur demande au moteur ; l'encodeur indique ce que le moteur fait réellement. La séance 6 réunit pour la première fois la chaîne d'action et la chaîne de mesure, sans encore fermer la boucle de contrôle.

Limite volontaire de la séance :
Aucune régulation de vitesse n'est réalisée ici. Les écarts entre moteurs, la compensation automatique et la boucle fermée seront étudiés plus tard dans le projet.

Prochaine étape :
Commencer la séance 7 avec la lecture des données brutes de l'accéléromètre et du gyroscope du MPU6050 via I2C, puis afficher les mesures sur l'UART.

Séance 7 – Lecture IMU brute

Journal de séance

Objectif : Lire les données brutes de l'IMU.

Technologie : I2C + MPU6050.

Réalisation :

- lecture des registres d'accélération ;
- lecture du gyroscope ;
- affichage UART.

Problème attendu :
Valeurs instables et bruitées.

Apprentissage :
Les capteurs bruts ne sont jamais directement exploitables pour le contrôle.

Séance 8 – Filtre complémentaire

Journal de séance

Date : Août–octobre 2026

Objectif :
Estimer l'angle de tangage du robot en fusionnant les mesures de l'accéléromètre et du gyroscope du MPU6050, puis disposer d'un module logiciel réutilisable pour une future boucle de régulation.

Objectif pédagogique :
Distinguer une mesure d'inclinaison absolue, bruitée mais sans dérive cumulative, d'une mesure de vitesse angulaire, fluide à court terme mais susceptible de dériver après intégration. Mesurer réellement le pas temporel, valider les signes des axes et comprendre la pondération d'un filtre complémentaire discret.

Configuration retenue :

- Carte `NUCLEO-F446RE`, programmation C bare-metal sans HAL.
- MPU6050 sur `I2C1`, broches `PB8/PB9`, adresse `0x68`.
- Tangage autour de l'axe Y ; accélérations X et Z pour l'angle absolu, vitesse gyroscopique Y pour l'intégration.
- Base de temps matérielle `TIM5`, compteur à 1 µs par tick sous l'hypothèse d'une horloge timer de 16 MHz.
- Affichage de diagnostic avec `USART2` à 115200 bauds, environ toutes les 100 ms, sans ralentir volontairement l'acquisition.
- FPU du Cortex-M4F activée avant les calculs flottants ; bibliothèque mathématique

liée pour `atan2f()`.

Réalisation :

1. Reprise des pilotes `uart`, `i2c` et `mpu6050` développés lors des séances précédentes.
2. Calcul de l'angle accélérométrique par $\theta_{acc} = \text{atan2}(-A_x, A_z) \times 180/\pi$, après vérification expérimentale de la convention d'axes.
3. Activation explicite de la FPU avec le registre `SCB_CPACR`, puis barrières d'instructions `DSB/ISB`.
4. Vérification de la base de temps TIM5 : mesure d'intervalles de l'ordre de 701663 µs lors d'un premier test à temporisation logicielle, puis lecture du temps entre acquisitions.
5. Calibration du biais du gyroscope au repos, par moyenne de plusieurs échantillons, avant chaque expérience.
6. Conversion de la mesure gyroscopique corrigée en degrés par seconde et intégration discrète : $\theta_{gyro,k} = \theta_{gyro,k-1} + \omega_y \Delta t$.
7. Test des signes : une diminution de l'angle ACC s'accompagne d'une vitesse Y principalement négative et d'une diminution de l'angle GYRO. Conservation de l'opérateur `+=`.
8. Réalisation du filtre complémentaire à coefficient fixe, initialement $\alpha = 0{,}98$.
9. Comparaison simultanée de quatre filtres : $\alpha = 0{,}90$, 0,95, 0,98 et 0,99, tous alimentés par les mêmes acquisitions.
10. Encapsulation du filtre dans `Inc/filtre_complementaire.h` et `Src/filtre_complementaire.c`, puis validation du programme principal avec ce module.

Problèmes rencontrés et solutions :

— **Blocage lors du calcul de `atan2f()` :** la FPU n'était pas encore activée dans le nouveau projet. Activation des coprocesseurs CP10/CP11 dans `SCB_CPACR` avant les opérations flottantes ; vérification du lien avec `-lm`.

— **Imprécision d'une temporisation par boucle :** utilisation du compteur matériel TIM5 et de la soustraction de deux timestamps non signés. La temporisation d'affichage UART est indépendante de la boucle de mesure.

— **Fluctuations et divergence des angles :** mesure séparée des angles ACC et GYRO, calibration du gyroscope immobile, puis comparaison expérimentale avant fusion.

— **Dépendance au pas d'échantillonnage :** dans la version finale modulaire, calcul du coefficient α à partir d'une constante de temps τ et du Δt mesuré, plutôt que conservation d'un coefficient fixe.

Résultats expérimentaux représentatifs :

— Au repos, les vitesses gyroscopiques corrigées sont majoritairement proches de zéro (par exemple $-0{,}145$, 0,015 et 0,328 degrés/s) ; l'angle intégré est resté autour de 10,50° sur un extrait, tandis que l'angle ACC fluctuait autour de 9° à 12°.

— Le temps entre deux acquisitions affiché pendant les essais est d'environ $\Delta t = 0{,}0017$ s, correspondant approximativement à 588 acquisitions/s ; il s'agit d'un affichage arrondi à quatre décimales.

— En rotation contrôlée, les variations des angles ACC et GYRO ont le même signe global : par exemple ACC passe de 28,57° à 11,42°, pendant que la vitesse Y reste principalement négative.

— Avec le filtre $\alpha = 0{,}98$, une phase de stabilisation montre ACC variant entre 11,32° et 13,58° alors que l'angle filtré reste proche de 12,6° à 13°.

— Dans le test comparatif, ACC vaut 24,17° après une mesure à 27,28°. Les quatre filtres affichent respectivement 24,93°, 25,20°, 25,48° et 25,58° : les coefficients plus élevés amortissent davantage ce changement rapide.

— En fin de comparaison, alors que le gyroscope intégré seul indique 19,12°, les filtres fournissent 34,57°, 34,49°, 34,39° et 34,33° face à ACC 35,42°, montrant la correction progressive de l'écart accumulé.

Comparaison des coefficients :

α	Influence de l'accéléromètre	Observation / compromis
0,90	Correction plus forte	Réagit davantage aux fluctuations ACC.
0,95	Correction intermédiaire	Compromis entre réactivité et lissage.
0,98	Correction plus progressive	Bon point de départ expérimental.
0,99	Correction la plus lente	Atténue davantage les perturbations rapides ACC.

La valeur 0,98 est conservée comme référence provisoire. Les essais ne déterminent pas encore un coefficient optimal pour le robot monté : une mesure de référence, des métriques de bruit et des essais dynamiques reproductibles seront nécessaires.

Organisation logicielle finale :

Fichier	Rôle
`Inc/filtre_complementaire.h`	Structure contenant l'angle estimé et la constante de temps ; déclarations d'initialisation et de mise à jour.
`Src/filtre_complementaire.c`	Prédiction gyroscopique, calcul de α et correction accélérométrique.
`Src/timebase.c`	Lecture du temps matériel avec TIM5.
`Src/main.c`	Initialisation, calibration, acquisitions I2C, calcul de Δt, appels du filtre et diagnostic UART.

Schéma-bloc de la chaîne de mesure :

$$\text{MPU6050} \longrightarrow \text{acquisitions I2C}$$

$$\underbrace{A_x, A_z \longrightarrow \theta_{acc}}_{\text{référence d'inclinaison}} \qquad \underbrace{G_y \longrightarrow \omega_y \Delta t}_{\text{variation rapide}}$$

$$\theta_{acc} + \omega_y \Delta t + \theta_{filtre,k-1} \longrightarrow \text{filtre complémentaire} \longrightarrow \theta_{filtre,k}$$

$$\text{TIM5} \longrightarrow \Delta t \qquad \text{UART} \longleftarrow \theta_{filtre,k}$$

Apprentissage principal :
L'accéléromètre limite la dérive à long terme, tandis que le gyroscope restitue rapidement les variations d'orientation. La fusion combine leurs avantages ; elle exige cependant une calibration, une convention de signe cohérente et un pas temporel correctement mesuré.

Résultat final :
L'estimation de tangage par filtre complémentaire est fonctionnelle et le module `filtre_complementaire.h/.c` a été testé avec succès sur la STM32F446RE. La mesure est prête à être intégrée aux futures expériences de contrôle, sous réserve de valider le comportement lorsque l'IMU sera fixée au châssis.

Prochaine étape :
Poursuivre avec les essais prévus dans la feuille de route, puis utiliser l'angle filtré comme retour de mesure lors de la future régulation d'équilibre.

Séance 9 – Test moteur en boucle ouverte

Journal de séance

Objectif : Tester les moteurs sans contrôle fermé.

Technologie : PWM fixe.

Réalisation :

- test à vitesse constante ;
- test sous différentes charges ;
- observation du comportement mécanique.

Problème attendu :
La vitesse varie selon la charge et l'état de la batterie.

Apprentissage :
La boucle ouverte est insuffisante pour un système dynamique précis.

Séance 10 – Boucle de contrôle vitesse

Journal de séance

Objectif : Stabiliser la vitesse moteur.

Technologie : PID vitesse + encodeurs.

Réalisation :

- implémentation d'un PID de vitesse ;
- lecture encodeur ;
- ajustement automatique du PWM.

Problème attendu :
Oscillations ou réponse trop lente.

Solution :
Régler progressivement les gains, en commençant par le gain proportionnel.

Apprentissage :
Un PID mal réglé crée de l'instabilité au lieu de la corriger.

Séance 11 – Boucle d'équilibre en simulation

Journal de séance

Objectif : Tester le contrôle d'équilibre avant l'intégration physique.

Technologie : Simulation logicielle d'un pendule inversé simplifié.

Réalisation :

- modélisation simplifiée de l'angle ;
- test du PID angle ;
- validation du comportement attendu.

Apprentissage :
Toujours tester un contrôle avant de l'appliquer directement à un système instable réel.

Séance 12 – Construction du châssis

Journal de séance

Objectif : Construire la base mécanique du robot.

Technologie : Mécanique + intégration électronique.

Réalisation :

- assemblage de la structure du robot ;
- fixation des moteurs ;
- positionnement de la batterie pour maîtriser le centre de gravité ;
- installation de la STM32 et du driver moteur ;
- câblage initial.

Points critiques :

- équilibre mécanique ;
- rigidité du châssis ;
- alignement des roues ;
- fixation stable de l'IMU.

Apprentissage :
La stabilité d'un robot commence en mécanique, pas en code.

Séance 13 – Première tentative d'équilibre

Journal de séance

Objectif : Faire tenir le robot debout.

Technologie : IMU + PID angle + PWM.

Réalisation :

— boucle de contrôle à 500–1000 Hz ;
— lecture IMU en temps réel ;
— correction moteur selon l'angle.

Résultat probable :
Oscillations rapides et instabilité initiale.

Apprentissage :
Un système instable doit être réglé, pas abandonné.

Séance 14 – Stabilisation progressive

Journal de séance

Objectif : Améliorer la stabilité du robot.

Actions :

— réglage progressif du PID ;
— réduction du bruit IMU ;
— amélioration de la fréquence de boucle ;
— observation du comportement mécanique.

Résultat attendu :
Réduction des oscillations et maintien de l'équilibre pendant plusieurs secondes.

Apprentissage :
Le contrôle est un processus itératif.

Séance 15 – Déplacement contrôlé

Journal de séance

Objectif : Faire avancer et reculer le robot sans tomber.

Technologie : Contrôle angle + consigne vitesse.

Réalisation :

— ajout d'une consigne de vitesse ;
— couplage entre équilibre et déplacement ;
— test de stabilité dynamique.

Apprentissage :
Stabilité et mouvement forment un système couplé.

Séance 16 – Préparation autonomie

Journal de séance

Objectif : Préparer une navigation simple d'un point A vers un point B.

Technologie : Odométrie + contrôle position.

Réalisation :

- estimation de la distance parcourue ;
- correction de trajectoire ;
- test d'un mouvement simple sur une distance donnée.

Apprentissage :
Un robot stable est une base pour l'autonomie.

5. Les composants

STM32 Nucleo-F446RE

Fiche composant

- Microcontrôleur STM32F446RE.
- Coeur ARM Cortex-M4.
- Timers adaptés au PWM et aux encodeurs.
- Interfaces UART, SPI, I2C.
- ADC, DMA et interruptions.
- ST-Link intégré pour programmation et debug.

IMU MPU6050

Fiche composant

- Accéléromètre 3 axes.
- Gyroscope 3 axes.
- Communication I2C.
- Utilisé pour estimer l'angle du robot.

Moteurs avec encodeurs

Fiche composant

- Moteurs DC avec réduction mécanique.
- Encodeurs quadrature voies A et B.
- Mesure de vitesse et de position.

— Utilisés pour le contrôle de vitesse et l'odométrie.

Driver moteur TB6612FNG

Fiche composant

— Double pont en H permettant de commander deux moteurs DC.
— Interface de puissance entre les signaux logiques de la STM32 et les moteurs.
— Alimentation logique `VCC` utilisée à 3,3 V dans le projet.
— Alimentation moteur `VM` séparée de la logique.
— Entrées `AIN1/AIN2` utilisées pour sélectionner le sens du moteur A.
— Entrée `PWMA` reliée à `PA6 / TIM3_CH1` pour recevoir le signal PWM.
— Entrée `STBY` utilisée pour activer ou désactiver le driver.
— Sorties `AO1/AO2` reliées aux deux fils de puissance du moteur.
— Lors de la séance 6, le canal A a été validé en marche avant, marche arrière, arrêt et variation de PWM.

Alimentation

Fiche composant

— Batterie LiPo 2S 7,4 V comme source d'énergie principale.
— Convertisseur Buck LM2596 utilisé pendant les essais pour abaisser la tension destinée au moteur à environ 5 V.
— Sortie du Buck reliée à `VM` du TB6612FNG pour l'étage de puissance moteur.
— Alimentation logique du TB6612FNG fournie en 3,3 V depuis la STM32.
— Masse commune entre STM32, TB6612FNG, Buck et encodeurs.
— Interrupteur ON/OFF sur la ligne positive.
— Connecteurs Deans pour la puissance.

6. Notions théoriques

Memory-Mapped I/O : carte mémoire, périphériques et registres

Dans un microcontrôleur STM32, l'espace d'adressage ne contient pas uniquement la mémoire dans laquelle sont stockées les variables. Une partie des adresses est réservée aux périphériques matériels : GPIO, UART, I2C, timers, ADC ou encore RCC. Ce principe est appelé *Memory-Mapped I/O*.

Pour le processeur, lire ou écrire un registre matériel ressemble donc à lire ou écrire une case mémoire. La différence est que cette opération produit un effet physique ou permet de lire l'état d'un périphérique.

Élément	Signification
Carte mémoire	Organisation de tout l'espace d'adresses vu par le processeur.
Périphérique	Bloc matériel, par exemple GPIOA ou USART2.
Adresse de base	Première adresse réservée au périphérique.
Offset	Position d'un registre par rapport à l'adresse de base.
Registre	Petite zone mémoire, souvent de 32 bits, qui configure ou décrit le périphérique.
Bit ou champ de bits	Partie précise d'un registre associée à une fonction.

La règle essentielle est :

Adresse du registre = adresse de base du périphérique + offset du registre

Exemple avec GPIOA :

— `GPIOA_BASE = 0x40020000` ;

— l'offset de `MODER` est `0x00` ;

— l'adresse de `GPIOA_MODER` est donc `0x40020000` ;

— l'offset de `ODR` est `0x14` ;

— l'adresse de `GPIOA_ODR` est donc `0x40020014`.

Exemple avec USART2 :

Registre	Offset	Rôle principal
`USART2_SR`	`0x00`	Indique l'état du périphérique, par exemple `TXE`.
`USART2_DR`	`0x04`	Contient la donnée à transmettre ou reçue.
`USART2_BRR`	`0x08`	Configure le débit en bauds.
`USART2_CR1`	`0x0C`	Active et configure les fonctions principales de l'USART.

Une définition comme :

```
#define USART2_DR (*(volatile uint32_t *)(USART2_BASE + 0x04UL))
```

peut être comprise en quatre étapes :

1. calculer l'adresse du registre avec la base et l'offset ;
2. considérer cette adresse comme un pointeur vers un entier de 32 bits ;
3. utiliser `volatile` pour imposer un véritable accès matériel ;
4. déréférencer le pointeur avec `*` pour lire ou modifier le registre.

Point d'attention

Une adresse incorrecte ou un mauvais bit peut configurer un autre périphérique ou produire un comportement imprévisible. Les adresses, offsets et positions des bits doivent toujours être vérifiés dans le manuel de référence et la datasheet du microcontrôleur.

Registres et opérations sur les bits

Un registre contient plusieurs fonctions indépendantes. Il ne faut donc généralement pas remplacer toute sa valeur lorsqu'on souhaite modifier un seul bit.

Les opérations les plus utilisées sont :

— activer un bit : `registre |= (1UL << n)` ;

— désactiver un bit : `registre &= ~(1UL << n)` ;

— tester un bit : `registre & (1UL << n)` ;

— inverser un bit : `registre ^= (1UL << n)` ;

— modifier un champ de plusieurs bits : effacer d'abord le champ avec un masque, puis écrire la nouvelle valeur.

Cette méthode permet de modifier uniquement la fonction recherchée sans perturber les autres bits du registre.

RCC et bus internes

Le RCC, ou *Reset and Clock Control*, distribue les horloges aux différents périphériques. Un périphérique dont l'horloge n'est pas activée ne peut pas fonctionner correctement, même si ses registres sont configurés.

Les périphériques sont reliés au processeur par plusieurs bus internes :

— AHB1 pour des périphériques rapides comme les GPIO ;

— APB1 pour plusieurs périphériques comme USART2, I2C et certains timers ;

— APB2 pour d'autres périphériques rapides comme certains USART et timers.

La démarche habituelle est donc : identifier le bus du périphérique, activer son horloge dans le registre RCC correspondant, puis configurer ses propres registres.

GPIO

Les GPIO sont des broches numériques configurables. Une broche peut notamment fonctionner comme entrée, sortie, fonction alternative ou entrée analogique. Le registre `MODER` choisit ce mode.

En sortie GPIO, le programme commande directement le niveau logique grâce au registre `ODR`. En fonction alternative, la broche est confiée à un autre périphérique matériel, par exemple un USART ou un timer.

Les GPIO seront utilisés pour les LEDs, les boutons, les signaux de direction des moteurs et plusieurs fonctions alternatives.

UART

L'UART est une communication série asynchrone. Elle utilise principalement une ligne `TX` pour transmettre et une ligne `RX` pour recevoir. Comme aucune horloge n'est transmise sur un fil séparé, les deux équipements doivent partager le même débit et le même format de trame.

La configuration `115200 8N1` signifie : 115200 symboles par seconde, 8 bits de données, aucune parité et 1 bit de stop.

Dans ce projet, USART2 transmet sur PA2. Les registres principaux utilisés sont :

— `BRR` pour le débit ;

— `CR1` pour activer l'émetteur et le périphérique ;

— `SR` pour consulter l'état de la transmission ;

— `DR` pour écrire le caractère à transmettre.

L'UART sert au debug et à la télémétrie. Il permet d'observer les mesures, les états et les erreurs internes du robot depuis un terminal PC.

I2C

L'I2C, ou *Inter-Integrated Circuit*, est un bus série synchrone utilisé pour relier un contrôleur à un ou plusieurs circuits intégrés. Il utilise seulement deux lignes partagées :

— `SCL`, ou *Serial Clock*, qui transporte l'horloge générée par le maître ;

— `SDA`, ou *Serial Data*, qui transporte les adresses, les données et les acquittements dans les deux directions.

Dans le projet, la STM32 est le maître du bus et le MPU6050 est un périphérique esclave. La STM32 décide du début et de la fin des échanges, génère l'horloge et choisit le composant avec lequel elle souhaite communiquer.

STM32 maître → `SCL` + `SDA` → MPU6050 esclave

Sorties open-drain et résistances de tirage

Les lignes I2C utilisent un fonctionnement *open-drain*. Un périphérique peut tirer la ligne vers la masse pour produire un zéro logique, mais il ne force pas directement un niveau haut. Pour produire un état logique 1, tous les composants relâchent la ligne et une résistance de tirage la ramène vers 3,3 V.

Action électrique	État obtenu
Un composant tire la ligne vers GND	Niveau logique 0
Tous les composants relâchent la ligne	La résistance de tirage produit le niveau logique 1

Ce principe permet à plusieurs composants de partager les mêmes fils sans qu'un périphérique force un niveau haut pendant qu'un autre force un niveau bas. Les modules MPU6050 possèdent souvent déjà des résistances de tirage, mais leur présence et leur valeur doivent être vérifiées lors de l'intégration matérielle.

Adresse du périphérique et adresse du registre

Deux niveaux d'adressage interviennent lors de la lecture du MPU6050 :

Information	Exemple	Rôle
Adresse I2C du périphérique	`0x68`	Sélectionner le MPU6050 parmi les composants présents sur le bus.
Adresse du registre interne	`0x75`	Sélectionner le registre `WHO_AM_I` à l'intérieur du MPU6050.
Valeur du registre	`0x68`	Identifier la famille du composant qui a répondu.

Connaître l'adresse théorique `0x68` ne prouve pas que le capteur fonctionne. La lecture de `WHO_AM_I` vérifie en même temps le câblage, la configuration logicielle, l'adressage, l'écriture de l'adresse interne et la réception d'une donnée.

L'adresse I2C du MPU6050 est exprimée sur 7 bits. Lors de la transmission, le contrôleur construit un octet contenant cette adresse dans les bits 7 à 1, puis ajoute le bit lecture/écriture en position 0 :

$$\texttt{adresse transmise} = (\texttt{adresse 7 bits} \ll 1) + \texttt{bit R/W}$$

Ainsi, pour l'adresse `0x68` :

— en écriture, l'octet transmis est `0xD0` ;

— en lecture, l'octet transmis est `0xD1`.

START, STOP, ACK et NACK

Une transaction I2C est structurée par plusieurs événements :

— `START` : le maître prend le contrôle du bus et commence une transaction ;

— adresse + bit `R/W` : le maître choisit le destinataire et indique le sens de l'échange ;

— `ACK` : le récepteur tire SDA à zéro pour confirmer la réception ;

— `NACK` : absence d'acquittement, utilisée pour signaler une erreur ou la fin d'une réception ;

— *Repeated START* : nouveau départ sans libérer le bus, utile pour passer d'une phase d'écriture à une phase de lecture ;

— `STOP` : le maître termine l'échange et libère le bus.

Pour lire un registre, la STM32 doit d'abord écrire son adresse interne, puis relancer immédiatement une transaction en lecture :

`START` → adresse périphérique + écriture → `ACK` → adresse du registre → *Repeated START* → adresse périphérique + lecture → `ACK` → donnée → `NACK` → `STOP`

Configuration de I2C1 sur STM32F446RE

Dans la séance 3, `I2C1` a été configuré avec `PB8` pour SCL et `PB9` pour SDA. Ces broches sont placées en fonction alternative `AF4`, en mode *open-drain*.

Les principaux registres utilisés sont :

Registre	Rôle
`CR1`	Activation du périphérique et génération de `START`, `STOP` et des acquittements.
`CR2`	Indication de la fréquence de l'horloge APB1 exprimée en MHz.
`CCR`	Réglage de la période de l'horloge SCL.
`TRISE`	Prise en compte du temps maximal de montée des lignes.
`DR`	Envoi de l'adresse, du numéro de registre ou d'une donnée ; réception d'une donnée.
`SR1`	Événements et erreurs : `SB`, `ADDR`, `BTF`, `TXE`, `RXNE`, `AF`.
`SR2`	État général du bus, notamment le bit `BUSY`.

Avec une horloge APB1 de 16 MHz et un bus standard à 100 kHz :

— `CR2.FREQ = 16;`

— `CCR = 16 000 000 / (2 × 100 000) = 80;`

— `TRISE = 16 + 1 = 17.`

Le champ `FREQ` ne reçoit donc pas la fréquence du bus I2C. Il reçoit la fréquence d'entrée du périphérique exprimée en MHz. Le registre `CCR` utilise ensuite cette information pour produire l'horloge SCL recherchée.

Validation progressive

La communication a été validée en deux étapes :

1. **Test d'adresse :** génération de `START`, envoi de l'adresse `0x68` et vérification du bit `ADDR`. La réponse `ACK` confirme qu'un composant est présent.
2. **Lecture de l'identifiant :** écriture de l'adresse interne `0x75`, *Repeated START*, lecture d'un octet et vérification de la valeur `0x68`.

Des compteurs de temporisation sont utilisés dans les boucles d'attente. Ils empêchent le programme de rester bloqué indéfiniment si le bus est occupé, si la condition `START` n'est pas générée ou si le périphérique ne répond pas.

Point d'attention

Une LED allumée sur le module prouve seulement que le circuit reçoit une alimentation. Elle ne confirme ni le câblage de SDA/SCL, ni la bonne adresse, ni la réussite des transactions. La validation logicielle par `ACK` puis par lecture de `WHO_AM_I` reste indispensable.

PWM et timers

Le PWM, ou *Pulse Width Modulation*, est un signal numérique périodique dont on fait varier la largeur de l'impulsion. La sortie alterne rapidement entre un niveau bas et un niveau haut, mais la proportion de temps passée à l'état haut peut être modifiée.

Cette proportion est appelée **rapport cyclique** :

```
D = T_haut / T_periode
```

Un rapport cyclique de 0 % signifie que la sortie reste toujours basse. À 50 %, elle reste haute pendant la moitié de chaque période. À 100 %, elle reste toujours haute.

Tension moyenne

Pour un signal compris entre 0 V et 3,3 V, la tension moyenne idéale vaut :

$$V_{moyenne} = 3{,}3 \times D$$

Par exemple, avec un rapport cyclique de 60 % :

$$V_{moyenne} = 3{,}3 \times 0{,}60 = 1{,}98\ V$$

C'est cette valeur moyenne qu'un multimètre en mode tension continue affiche approximativement. Un oscilloscope ou un analyseur logique serait nécessaire pour observer directement les niveaux hauts et bas, la période, la fréquence et la largeur exacte des impulsions.

Génération matérielle avec un timer

La STM32 ne génère pas le PWM en exécutant continuellement des instructions pour mettre une broche à zéro puis à un. Elle configure un timer matériel qui travaille ensuite de manière autonome.

Horloge du timer → `PSC` → compteur `CNT` → comparaison avec `CCR1` → canal `TIM3_CH1` → AF2 → PA6

Les registres principaux sont :

Registre	Rôle
`PSC`	Divise la fréquence d'entrée du timer. La division réelle vaut `PSC + 1`.
`CNT`	Contient la valeur courante du compteur.
`ARR`	Fixe la valeur maximale du compteur et donc la période.
`CCR1`	Fixe la valeur de comparaison du canal 1 et donc le rapport cyclique.
`CCMR1`	Configure le mode de sortie du canal, notamment PWM mode 1.
`CCER`	Active la sortie du canal et règle sa polarité.
`CR1`	Démarre le compteur et active notamment le preload de `ARR`.
`EGR`	Permet de générer un événement de mise à jour avec le bit `UG`.

Fréquence du compteur et fréquence PWM

$$f_{compteur} = f_{timer} / (PSC + 1)$$

$$f_{PWM} = f_{timer} / ((PSC + 1) \times (ARR + 1))$$

Le terme `ARR + 1` apparaît parce que le compteur parcourt toutes les valeurs de 0 à `ARR`.

Dans la séance 4 :

— `f_timer = 16 MHz` ;

— `PSC = 15`, donc le compteur fonctionne à 1 MHz ;

— `ARR = 999`, donc une période contient 1000 coups de compteur ;

— `f_PWM = 1 000 000 / 1000 = 1000 Hz`.

La période vaut donc 1 ms.

Rapport cyclique et registre CCR1

En PWM mode 1, la sortie est active lorsque `CNT < CCR1`. Elle devient inactive lorsque le compteur atteint ou dépasse `CCR1`. Le rapport cyclique idéal est donc :

```
D = CCR1 / (ARR + 1)
```

Avec `ARR = 999` :

Rapport cyclique	Valeur de `CCR1`	Temps haut à 1 kHz
0 %	0	0 µs
25 %	250	250 µs
60 %	600	600 µs
100 %	1000	1000 µs

La fonction du pilote applique directement cette relation :

```
CCR1 = ((ARR + 1) × duty_percent) / 100
```

Le pourcentage est limité à 100 afin d'empêcher l'écriture d'une commande incohérente.

Preload et mise à jour

Les registres `ARR` et `CCR1` peuvent utiliser des registres de preload. Une nouvelle valeur est alors stockée temporairement et appliquée lors du prochain événement de mise à jour. Cela évite de modifier une période en plein milieu et de créer une impulsion irrégulière.

Le bit `ARPE` active le preload de `ARR`, tandis que `OC1PE` active celui de `CCR1`. Le bit `UG` du registre `EGR` force un événement de mise à jour, notamment pour charger immédiatement les valeurs de `PSC`, `ARR` et `CCR1` avant le démarrage.

Fonction alternative et sortie physique

La sortie produite par `TIM3_CH1` reste interne au microcontrôleur tant qu'elle n'est pas reliée à une broche. La broche `PA6` est donc configurée en fonction alternative `AF2`.

Timer configuré → canal 1 activé → `TIM3_CH1` → fonction alternative AF2 → sortie physique PA6

PWM et commande moteur

Le PWM produit par la STM32 est un signal logique de commande. La broche `PA6` ne fournit pas directement l'énergie nécessaire au moteur : elle transmet seulement l'information de commande au TB6612FNG.

Depuis la séance 6, la chaîne réellement utilisée est :

STM32 / `TIM3_CH1` / PA6 → `PWMA` → TB6612FNG → alimentation moteur `VM` → `AO1/AO2` → moteur

Le rapport cyclique détermine le niveau de commande appliqué au moteur. Une augmentation du rapport cyclique entraîne généralement une augmentation de la vitesse, mais le pourcentage de PWM n'est pas égal au pourcentage de la vitesse maximale. Les frottements, la charge, l'alimentation, le réducteur et la dynamique mécanique influencent le résultat réel.

Les essais de la séance 6 l'ont montré directement : pour le premier moteur, des commandes de 20 %, 40 %, 60 % et 80 % ont conduit respectivement à des vitesses stabilisées d'environ 20, 47, 72–73 et 96–97 tr/min en valeur absolue.

Le PWM représente donc la **commande**, tandis que l'encodeur fournit la **mesure réelle**.

Pont en H et TB6612FNG

Un moteur DC change de sens lorsque la polarité appliquée à ses deux bornes est inversée. Le pont en H du TB6612FNG réalise cette inversion électroniquement, sans modifier physiquement les fils du moteur.

Pour le canal A utilisé pendant la séance 6 :

— `AIN1` est reliée à `PC0` ;

— `AIN2` est reliée à `PC1` ;

— `STBY` est reliée à `PC2` ;

— `PWMA` est reliée à `PA6 / TIM3_CH1` ;

— `AO1` et `AO2` sont reliées aux deux fils de puissance du moteur.

La séparation fonctionnelle peut être résumée ainsi :

Signal	Fonction
`STBY`	Active ou désactive le driver.
`AIN1/AIN2`	Déterminent le sens de rotation choisi.
`PWMA`	Reçoit le PWM et règle le niveau de commande du moteur.
`AO1/AO2`	Appliquent la puissance commutée aux deux bornes du moteur.

Dans la convention logicielle retenue :

`AIN1`	`AIN2`	Commande
1	0	`moteur_avant()`
0	1	`moteur_arriere()`
0	0	état utilisé par `moteur_arreter()`

Le changement de sens ne nécessite donc pas de modifier le rapport cyclique. À PWM identique, inverser `AIN1` et `AIN2` inverse le sens du moteur.

Alimentation logique, alimentation moteur et masse commune

Le TB6612FNG sépare la partie logique de la partie puissance :

- `VCC` alimente la logique du driver et est reliée au 3,3 V de la STM32 ;
- `VM` alimente l'étage moteur et a été alimentée à environ 5 V pendant les essais ;
- les masses de la STM32, du driver et de l'alimentation moteur sont reliées ensemble.

Le convertisseur Buck LM2596 a été utilisé pour abaisser la tension de la LiPo 2S avant de l'appliquer à `VM`. La sortie du Buck a d'abord été réglée au multimètre, sans moteur connecté, puis vérifiée à environ 5,0 V.

La masse commune est indispensable : un niveau logique de 3,3 V n'a de sens pour le TB6612FNG que si la STM32 et le driver utilisent le même potentiel de référence.

LiPo 2S → Buck 5 V → `VM` → TB6612FNG → moteur

STM32 3,3 V → `VCC` GND STM32 = GND TB6612FNG = GND Buck

Commande et retour de mesure

La séance 6 réunit deux chaînes jusque-là étudiées séparément :

Action : STM32 → direction + PWM → TB6612FNG → moteur

Mesure : moteur → encodeur A/B → TIM2 → position + TIM5 → Δt → vitesse

À PWM constant de 60 %, l'inversion du sens du moteur provoque aussi l'inversion du signe de ΔCNT et donc du signe des RPM. Dans le câblage actuel, la commande `AVANT` produit un comptage négatif et la commande `ARRIERE` un comptage positif.

Cette distinction entre commande et mesure est fondamentale pour la suite : une future boucle de régulation pourra comparer une consigne de vitesse à la vitesse réellement mesurée puis ajuster automatiquement le PWM. Cette régulation n'est cependant pas réalisée pendant la séance 6.

Point d'attention

Une tension moyenne correcte au multimètre confirme le rapport cyclique de manière indirecte, mais elle ne suffit pas à vérifier précisément la fréquence, les fronts ou d'éventuelles déformations du signal. Une mesure à l'oscilloscope reste la validation la plus complète.

Encodeurs

Un encodeur incrémental transforme une rotation mécanique en une suite de transitions numériques. Dans le projet, les motoréducteurs utilisent un encodeur en quadrature composé de deux voies, A et B, déphasées l'une par rapport à l'autre.

Pourquoi deux voies ?

Avec une seule voie, il est possible de compter des impulsions et donc d'estimer un déplacement. Avec deux voies déphasées, l'ordre des transitions permet en plus de connaître le sens de rotation.

Une rotation peut produire la séquence :

`00` → `10` → `11` → `01` → `00`

Dans l'autre sens, l'ordre est inversé :

`00` → `01` → `11` → `10` → `00`

Le sens n'est donc pas déduit de la valeur instantanée de A ou B prise séparément, mais de la succession des états des deux voies.

Timer en mode encodeur

Sur STM32, un timer peut utiliser directement les deux entrées de capture comme interface encodeur. Dans la séance 5, les voies sont reliées à `TIM2_CH1` et `TIM2_CH2` :

Voie A → PA0 / AF1 → TIM2_CH1 / TI1
Voie B → PA1 / AF1 → TIM2_CH2 / TI2

TI1 + TI2 → logique quadrature → `TIM2_CNT`

Les registres principaux utilisés sont :

Registre	Rôle
`CCMR1`	Configure CH1 et CH2 comme entrées reliées à TI1 et TI2.
`CCER`	Définit notamment la polarité des deux entrées.
`SMCR`	Sélectionne le mode encodeur avec le champ `SMS`.
`CNT`	Contient la position relative courante sous forme de comptes.
`ARR`	Fixe la plage maximale du compteur.
`CR1`	Le bit `CEN` démarre ou arrête le compteur.

En mode encodeur, il faut retenir une différence fondamentale avec le PWM :

Utilisation du timer	Ce qui fait évoluer `CNT`
PWM / base de temps	L'horloge du timer, éventuellement divisée par `PSC`.
Mode encodeur	Les transitions détectées sur les voies A et B.

Le processeur configure donc le périphérique une seule fois, puis le timer compte matériellement les transitions sans qu'une interruption logicielle soit nécessaire pour chaque front.

Position, angle et calibration

La valeur brute de `CNT` représente des comptes, et non directement des degrés ou des tours. Il faut connaître le nombre de comptes correspondant à un tour complet de l'axe.

Une calibration expérimentale sur trois séries de dix tours a donné une moyenne d'environ 3881,47 comptes par tour. La constante utilisée dans le projet est arrondie à :

```
ENCODER_COUNTS_PER_REV = 3881
```

Le nombre de tours cumulés vaut alors :

$$N_{tours} = \frac{CNT}{3881}$$

et l'angle cumulé :

$$\theta = \frac{CNT}{3881} \times 360^\circ$$

Cette position est relative au zéro choisi au démarrage. Une rotation dans le sens opposé produit des comptes négatifs lorsqu'ils sont interprétés côté logiciel avec un type signé.

Vitesse à partir de la variation de position

Une valeur unique de `CNT` ne donne pas la vitesse. Il faut comparer deux lectures séparées dans le temps :

$$\Delta CNT = CNT_2 - CNT_1$$

$$\Delta t = t_2 - t_1$$

Avec une base de temps exprimée en microsecondes, la vitesse en tours par minute est calculée par :

$$RPM = \frac{\Delta CNT \times 60 \times 1\,000\,000}{3881 \times \Delta t_{\mu s}}$$

Le signe de ΔCNT est conservé : une vitesse positive et une vitesse négative correspondent donc aux deux sens de rotation.

Base de temps indépendante avec TIM5

La temporisation par boucle utilisée lors des premiers tests n'est pas suffisamment précise pour mesurer une vitesse. TIM5 est donc configuré comme chronomètre matériel.

Avec une horloge de 16 MHz et `PSC = 15` :

```
16 MHz / (15 + 1) = 1 MHz
```

Ainsi :

```
1 tick TIM5 = 1 µs
```

TIM2 mesure donc le **mouvement**, tandis que TIM5 mesure le **temps**. La vitesse est obtenue en combinant les deux informations.

TIM2 $\rightarrow \Delta CNT$ TIM5 $\rightarrow \Delta t$
$\Delta CNT + \Delta t \rightarrow$ vitesse en tr/min

Distance et vitesse linéaire avec une roue

Si une roue de diamètre D est montée sur l'axe, sa circonférence vaut :

$$C = \pi D$$

Sans glissement, chaque tour complet déplace théoriquement le robot d'une circonférence. La distance parcourue peut donc être estimée par :

$$d = \frac{CNT}{3881} \times \pi D$$

Si D est exprimé en mètres, la vitesse linéaire en mètres par seconde est :

$$v = \frac{\pi D \times RPM}{60}$$

Le facteur 60 est nécessaire parce que les RPM sont exprimés en tours par minute. Ces relations serviront plus tard pour l'odométrie et l'estimation du déplacement du robot.

Point d'attention

La conversion encodeur → distance suppose que la roue roule sans glisser. Sur un robot réel, le patinage, la déformation des pneus et les différences entre les deux moteurs peuvent introduire une erreur d'odométrie. Une calibration expérimentale de chaque roue sera donc préférable lors de l'intégration finale.

Filtre complémentaire

Deux capteurs, deux comportements

L'accéléromètre mesure la force spécifique sur trois axes. Lorsqu'il est presque immobile, la projection de la gravité permet d'estimer une inclinaison absolue. Cette estimation est néanmoins sensible au bruit, aux vibrations et aux accélérations de translation. Le gyroscope mesure une vitesse angulaire, qui permet de reconstruire les changements d'angle à court terme mais dont l'intégration accumule les petites erreurs de biais.

Capteur	Avantage principal	Limite principale
Accéléromètre	Référence d'inclinaison à long terme.	Bruit et perturbations dynamiques.
Gyroscope	Mesure fluide des rotations rapides.	Dérive de l'angle intégré.

Angle de tangage avec l'accéléromètre

Avec la convention expérimentale utilisée pour une rotation autour de l'axe Y du MPU6050 :

$$\theta_{acc} = \operatorname{atan2}(-A_x, A_z)\frac{180}{\pi}$$

La fonction `atan2f()` renvoie un angle en radians en tenant compte du quadrant. La multiplication par $180/\pi$ permet de travailler en degrés. Les accélérations X et Z peuvent ici être utilisées dans la même unité, car leur facteur de conversion commun s'annule dans le calcul de l'angle.

Intégration du gyroscope et dérive

Après soustraction de son biais mesuré au repos, la vitesse angulaire sur l'axe Y est exprimée en degrés par seconde. L'intégration discrète est :

$$\theta_{gyro,k} = \theta_{gyro,k-1} + \omega_{y,k}\Delta t_k$$

Par exemple, 30°/s pendant 0,02 s correspondent à une variation de 0,6°. En revanche, un biais constant résiduel de 0,5°/s créerait une erreur d'environ 30° après une minute : c'est la dérive gyroscopique.

Pourquoi une base de temps matérielle ?

Le terme $\omega_y \Delta t$ doit utiliser un intervalle de temps réel, et non une durée supposée de boucle. TIM5 est configuré pour compter à 1 MHz à partir d'une horloge timer de 16 MHz avec `PSC = 15`. Le logiciel lit deux timestamps :

$$\Delta t_k = \frac{t_k - t_{k-1}}{10^6}$$

Les temps sont mesurés en microsecondes avec un compteur 32 bits non signé ; leur soustraction reste adaptée au débordement du compteur pour des écarts suffisamment courts. Les essais ont affiché environ 1,7 ms par acquisition. L'UART n'est sollicité qu'environ toutes les 100 ms afin de ne pas ralentir inutilement l'acquisition.

Fusion complémentaire : prédiction et correction

À chaque acquisition, le filtre commence par prévoir le nouvel angle à partir du gyroscope, puis le rapproche de l'angle accélérométrique :

$$\theta_{pred,k} = \theta_{filtre,k-1} + \omega_{y,k}\Delta t_k$$

$$\boxed{\theta_{filtre,k} = \alpha\,\theta_{pred,k} + (1-\alpha)\,\theta_{acc,k}}$$

Le terme α pèse la prédiction gyroscopique ; le terme $1-\alpha$ pèse la correction accélérométrique. Un α proche de 1 privilégie le comportement fluide à court terme du gyroscope, mais corrige la dérive plus lentement. Un α plus faible corrige plus vite, mais transmet davantage les fluctuations de l'accéléromètre.

Gyroscope $\longrightarrow$ $\omega_y \Delta t$ $\longrightarrow$ prédiction avec l'angle filtré précédent

Accéléromètre $\longrightarrow$ atan2 $\longrightarrow$ θ_{acc}

prédiction $\times\ \alpha + \theta_{acc} \times (1-\alpha)$ $\longrightarrow$ $\boxed{\theta_{filtre}}$

Constante de temps et fréquence d'échantillonnage

Avec un coefficient fixe, le comportement du filtre change si la fréquence d'acquisition varie. La version modulaire utilise donc une constante de temps τ exprimée en secondes :

$$\alpha_k = \frac{\tau}{\tau + \Delta t_k}$$

Cette discrétisation donne une correction dépendant du temps réellement écoulé. Pour retrouver environ $\alpha = 0{,}98$ avec $\Delta t = 0{,}0017$ s :

$$\tau \approx \frac{0{,}98}{1 - 0{,}98} \times 0{,}0017 \approx 0{,}083 \text{ s}$$

Cette constante de temps, choisie initialement pour reproduire les essais, n'est pas encore une valeur optimisée pour la dynamique du robot.

Architecture logicielle et exploitation future

Le type `FiltreComplementaire` conserve l'angle estimé et la constante de temps. La fonction `filtre_complementaire_init()` initialise l'état avec le premier angle ACC, et `filtre_complementaire_update` réalise la prédiction puis la correction. Le programme principal reste responsable de la lecture I2C, du calcul de Δt et de l'affichage UART.

L'angle filtré pourra servir de mesure pour une régulation d'équilibre. Il devra cependant être vérifié après fixation mécanique du MPU6050, avec une fréquence de boucle maîtrisée et des tests de sécurité avant commande effective des moteurs.

PID

Le PID corrige l'erreur entre une consigne et une mesure :

- le terme proportionnel réagit à l'erreur actuelle ;
- le terme intégral corrige une erreur persistante ;
- le terme dérivé anticipe les variations rapides de l'erreur.

Dans ce projet, un PID sera utilisé pour contrôler l'angle du robot et un autre pourra être utilisé pour la vitesse des moteurs. Les gains devront être réglés progressivement afin d'éviter les oscillations et l'instabilité.

7. Architecture du robot

Architecture matérielle

Élément	Rôle
STM32 Nucleo-F446RE	Contrôle temps réel, lecture capteurs, PID, PWM
MPU6050	Mesure d'accélération et vitesse angulaire
TB6612FNG	Interface de puissance entre STM32 et moteurs
Moteurs + encodeurs	Action mécanique + mesure du mouvement
LiPo 2S	Source d'énergie principale
Buck LM2596	Abaissement de la tension d'alimentation utilisée pour les moteurs pendant les essais
Châssis PVC expansé	Structure mécanique du robot

Chaîne moteur validée pendant la séance 6

La chaîne de commande d'un moteur est désormais validée expérimentalement :

STM32 → `PC0/PC1` : direction `PC2` : activation `PA6/TIM3_CH1` : PWM → TB6612FNG → moteur

L'alimentation de puissance utilisée pendant les essais suit le chemin :

`LiPo 2S` → `Buck` ≈ `5 V` → `VM` → `TB6612FNG` → `moteur`

Le retour de mesure reste indépendant :

Moteur → encodeur A/B → PA0/PA1 → TIM2 → position
TIM2 + TIM5 → vitesse réelle en tr/min

Principe de fonctionnement

— L'IMU mesure l'inclinaison du robot.

— La STM32 estime l'angle avec un filtre complémentaire.

— Le PID calcule la correction nécessaire.

— La STM32 génère les signaux PWM.

— Le driver TB6612FNG applique la puissance aux moteurs.

— Les encodeurs mesurent le mouvement réel.

8. Les erreurs et leçons apprises

Erreur 1 – PID instable

Cause possible : gain proportionnel trop élevé.

Solution : réduire le gain proportionnel puis ajouter progressivement les autres termes.

Leçon : régler P avant I et D.

Erreur 2 – Robot qui tremble

Cause possible : bruit IMU, câbles moteurs trop proches des signaux, châssis flexible.

Solution : filtrage, séparation des câbles puissance/signaux, fixation rigide de l'IMU.

Leçon : un problème de stabilité n'est pas toujours un problème de code.

Erreur 3 – Reset de la STM32

Cause possible : chute de tension lors de l'appel de courant des moteurs.

Solution : alimentation logique séparée via Buck Converter et masse commune propre.

Leçon : l'alimentation est une partie critique du système.

Erreur 4 – Réglage du Buck qui semble ne pas réagir

Observation : pendant la séance 6, la tension de sortie du LM2596 semblait initialement ne pas varier lorsque le potentiomètre était tourné.

Cause : le potentiomètre de réglage est multi-tours ; plusieurs rotations peuvent être nécessaires avant d'observer une variation sensible.

Solution : régler le Buck sans moteur connecté, mesurer en permanence la tension entre `OUT+` et `OUT-`, puis arrêter le réglage lorsque la valeur recherchée est atteinte. Pour les essais de la séance 6, la sortie a été réglée à environ 5,0 V.

Leçon : toujours vérifier au multimètre la sortie d'un convertisseur avant de connecter l'étage de puissance.

9. Améliorations futures

— ESP32 pour Wi-Fi et télémétrie.

— Interface web pour visualiser les données en temps réel.

— Réglage PID à distance.

— Écran OLED pour afficher l'état du robot.

— Capteurs de distance VL53L0X.

— Caméra.

— Navigation autonome.

— SLAM simple.

— Passage à un châssis imprimé en 3D ou aluminium.

10. Matériel et plan d'achat

Matériel acheté pour la version 1

Composant	Rôle
STM32 Nucleo-F446RE	Carte principale temps réel
MPU6050	Mesure inertielle
TB6612FNG	Driver moteur
Deux motoréducteurs avec encodeurs	Actionnement et mesure de mouvement
LiPo 2S 7,4 V – 2200 mAh	Batterie principale
Chargeur LiPo avec équilibrage	Recharge sécurisée
Buck LM2596	Abaissement de la tension pour l'alimentation moteur pendant les essais
Connecteurs Deans + câbles silicone	Distribution puissance
Interrupteur ON/OFF	Coupure alimentation
Multimètre	Diagnostic électrique
Cutter	Fabrication châssis
PVC expansé	Structure mécanique
Fils Dupont et câbles	Connexions logiques et puissance

Budget

Le coût total de la version initiale du projet est estimé à :

200 €

Achats reportés

— ESP32 pour Wi-Fi ;

— écran OLED ;

— capteurs de distance ;

— caméra ;

— châssis avancé.

Conclusion

Pourvu qu'aujourd'hui soit mieux qu'hier

Je ne cherche pas à être parfait.

Je cherche simplement à progresser.

Chaque ligne de code écrite.

Chaque bug corrigé.

Chaque composant compris.

Chaque test réalisé.

Me rapproche un peu plus de l'ingénieur que je veux devenir.

Le robot n'est pas la destination.

Le véritable projet, c'est moi.